

First Module Exercises

First challenge

Create a function that receives an enumerator as a parameter with the following options: `.addition`, `.subtraction`, `.multiplication`, `.division` and an array of numbers. Return the result after applying the selected operation to all numbers. Handle division by zero with `Result` or throws.

Example:

```
operation(.addition, numbers: [1,2,3,4]) // 10  
operation(.division, numbers: [20, 5, 0]) // error: division by zero
```

Second challenge

Create a function that receives an array of integers and returns a dictionary with the count of positive, negative and zero values.

Example:

```
checkNumbers([-3, 0, 2, 5, -1]) // ["positives": 2, "negatives": 2, "zeros":1]
```

Third challenge

Create a function that receives an integer and a limit number. Return an array of Strings with the multiplication table of the given number up to the limit.

Example:

```
multiply(number: 7, limit: 12) //  
["7x1=7",  
"7x2=14",  
...,  
"7x12=84"]
```

Fourth challenge

Make a function that receives a string that simulates a password and returns an enum PasswordStrength: .weak, .medium, .strong depending on rules: - Minimum 6 characters - Contains a capital letter - Contains a number - Contains a point - Bonus: contains a special symbol (!, @, #, etc.).

Example:

```
checkPassword("Pass123.") // .medium
checkPassword("Strong#Pass1.") // .strong
```

Fifth challenge

Make a function that receives an array of optional strings and returns all the non-nil values in a new array. If all values are nil, return a message: "All values are empty".

Example:

```
let texts: [String?] = ["Hello", nil, "World"]
filterOptionals(texts) // ["Hello", "World"]
```

Sixth challenge

Given an optional string, print "It's empty! You ripped me off!" when it's nil, "Thanks for my cat Schrödinger!" if input is "🐱" and "This ain't a cat!" when none of the others

Example:

```
receiveBox(with: "🐱") // "Thanks for my cat Schrödinger!"
receiveBox(with: nil) // "It's empty! You ripped me off!"
receiveBox(with: "lol") // "This ain't a cat!"
```

Seventh challenge

Create a function that simulates a simple bank account system.

Requirements:

Define a struct BankAccount with:

- owner: String

- balance: Double

Add methods:

- deposit(amount: Double) → increases the balance.
- withdraw(amount: Double) → decreases the balance, but only if there are sufficient funds. Otherwise, print "Insufficient funds".
- transfer(amount: Double, to otherAccount: inout BankAccount) → transfers money to another account if balance is enough.

Example:

```
var account1 = BankAccount(owner: "Alice", balance: 1000)
var account2 = BankAccount(owner: "Bob", balance: 500)
```

```
account1.withdraw(amount: 200) // balance = 800
account1.transfer(amount: 300, to: &account2)
```

```
// account1.balance = 500
// account2.balance = 800
```

Eight challenge

Implement the *Sieve of Eratosthenes* algorithm in Swift from the provided unit tests. Return and print the resulting array containing only the prime numbers within the range.

- The only parameter is the max number.
- 0 and 1 are not prime numbers.
- The next available number is marked as prime but the next multiples up to the max number are not.
- Find the next prime number and repeat the previous step until reaching max number.

Example:

```
PrimeCalculator.calculate(upTo: 10) // [2,3,5,7]
```

```
PrimeCalculator.calculate(upTo: 50) // [2,3,5,7,11,13,17,19,23,29,31,37,41,43,47]
```

```
PrimeCalculator.calculate(upTo: 85)  
// [2,3,5,7,11,13,17,19,23,29,31,37,41,43,47,53,59,61,67,71,73,79,83]
```

Ninth challenge

Given an array of people, find the youngest, the oldest and the difference in age between them, and return their respective ages and the age difference.

```
struct Person {  
  let name: String  
  let age: Int  
}
```

```
let son = Person(name: "Juan", age: 19)  
let daughter = Person(name: "Maria", age: 12)  
let mother = Person(name: "Benita", age: 60)  
let father = Person(name: "Camilo", age: 58)  
let family = [daughter, son, mother, father]
```

Example:

```
findAgeDifference(for: family) // (oldest: 60, youngest: 12, ageDifference: 48)
```

Tenth challenge

Write a closure that takes an array of optional strings and returns a new string that is the concatenation of all the strings in the array. Remember to use the methods map, filter and reduce to find the solution.

Example:

```
let someStrings: [String?] = ["This", "is", nil, "not", nil, "a", "drill", nil, "!!"]  
// This is not a drill!
```